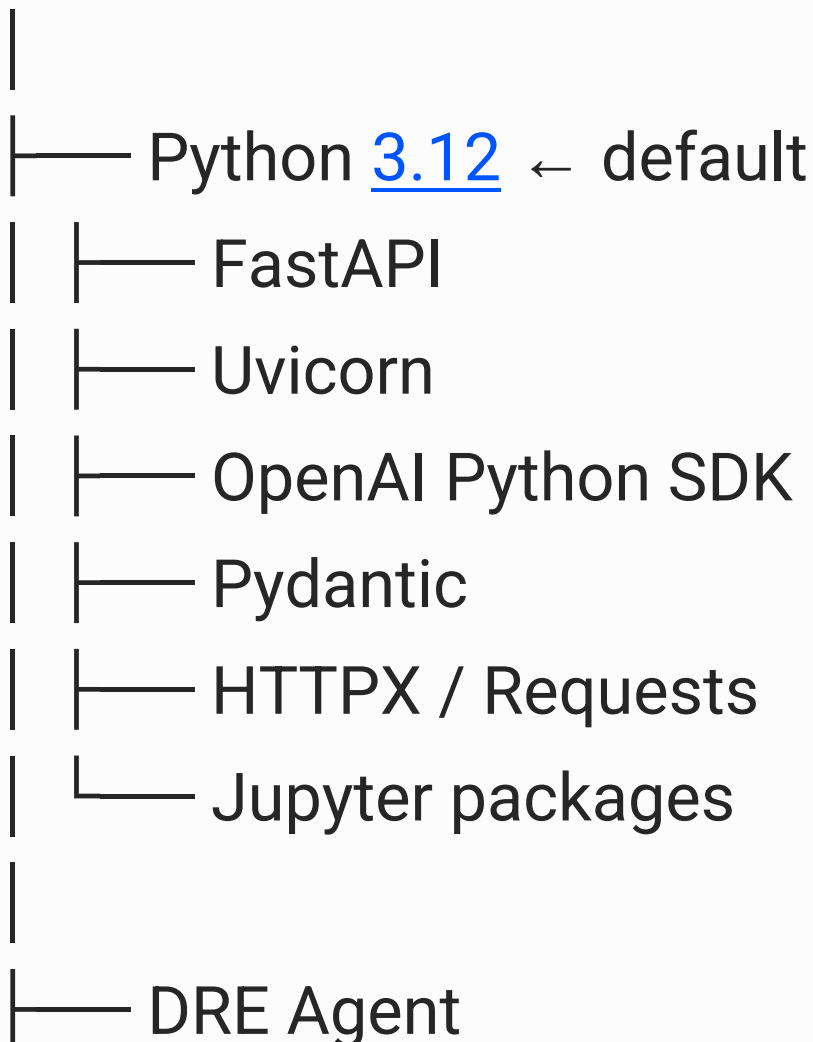


Worked for 14s

Yep. Here's the current software/runtime schematic only — focused on what matters when deciding what new software can run on this server.

Current DRE.SMALL software stack

Ubuntu [24.04](#)



- | | — FastAPI application
- | | — OpenAI Responses API
- | | — shell-command tool
- | | — Tiny HTML Web UI
- | | — Port 8000
- |
- | — Nginx
- | | — reverse proxy
- | | — currently exposes/proxies several

RunPod template ports

- |
- | — RunPod Web Terminal
- | | — gotty :19123
- |
- | — Git
- |
- | — GitHub CLI gh
- | | — just installed; authentication still needs

verification

|

|—— Shell / build environment

| |—— Bash

| |—— curl / wget

| |—— jq

| |—— GCC / G++

| |—— CMake

| |—— Git

| |—— tmux

| |—— nano / vim

|
|—— Linux service environment

|—— NO systemd as PID 1

Important installed-but-not-running software

These exist but are not currently active services:

Software State Why it matters

JupyterLab / Notebook Installed, off Could

provide browser-based Python/dev environment

OpenSSH server Installed, off/not configured

Could provide SSH

Nginx Running Useful for new web apps/APIs

Python tooling Installed Excellent for adding

FastAPI services, memory loaders, SSE,

WebSockets

Important software NOT currently installed

This is probably the most useful part for
planning:

NO Ollama

NO llama.cpp

NO vLLM

NO Transformers

NO PyTorch

NO Open WebUI

NO Qdrant

NO Chroma

NO FAISS

NO PostgreSQL

NO Redis

NO Docker CLI

NO Podman

NO Node.js

NO npm

NO Go

NO Rust

NO Java

NO local model weights

NO embedding model

So right now this machine is essentially:

Linux

+

Python web/API environment

+

DRE Agent

+

OpenAI cloud LLM

+

Nginx

+

browser terminal

+

Git/GitHub tooling

What that means for future software

Very easy to add right now: Python applications,

FastAPI APIs, memory loaders, SQLite, SSE/live activity feeds, WebSockets, Git-based memory, background Python services.

Needs additional runtime installed: Open WebUI, Ollama/local LLMs, Node-based applications, Docker-based applications, vector databases depending on deployment method.

One architectural detail to remember: this is a Docker-style RunPod container, not a normal VM with systemd. Software expecting systemctl enable ... and traditional Linux boot services may need its startup handled by your container startup script instead.

That's the software baseline I would use whenever we're evaluating: "Can application X run on DRE.SMALL, and what would we have to add?"

Here's the practical “schematic” of Runpod networking + ports you'll want if you're building an app that shows a UI for a server running inside a Pod.

1) Mental model: what you're exposing

A Runpod Pod is essentially your container running somewhere with:

- Internal ports: what your process listens on inside the container (e.g. [0.0.0.0:3000](#))
- Exposure method: how Runpod makes that reachable from the internet

There are two main exposure paths you'll use:

2) Path A – HTTP Proxy (best for web UIs + REST)

Network path (high-level)

Browser/App → Cloudflare → Runpod proxy/LB
→ Pod internal port

How you “wire” it

- In your Pod/Template config, expose a port as <port>/http (example: 3000/http)
- Your service must listen on that internal port, and bind [0.0.0.0](#) (not [127.0.0.1](#))

The URL format (stable)

https://POD_ID-INTERNAL_PORT.proxy.runpod.net

Example:

<https://abc123xyz-3000.proxy.runpod.net>

Important behavior/limits (affects app design)

- HTTPS is handled for you (good for UIs).
- There's a ~100 second timeout at Cloudflare if the request doesn't begin responding/streaming in time. This matters for long inference requests or slow page loads.

When to use

- Web UI (React/Next/Vite, Gradio, Streamlit, etc.)
- REST APIs where requests complete quickly or stream output

3) Path B — Direct TCP (best for raw TCP, SSH, long-lived connections)

Network path (high-level)

Client → Public IP + mapped external port →
Pod internal TCP port

How you “wire” it

- Expose a port as <port>/tcp (example: 22/tcp, 8765/tcp)
- Runpod will assign:
 - a public IP
 - an external port that maps to your internal port

- You retrieve this from the Pod's Connect menu (or via Pod details API/SDK).

Key gotcha for app builders

The external port is usually not the same as the internal port. So if you're writing an app that auto-connects, you must discover the mapping, not assume external == internal.

Programmatic discovery

Runpod provides env vars like:

- RUNPOD_PUBLIC_IP
- RUNPOD_TCP_PORT_`<internalPort>`

So if you expose internal port 22/tcp, you'll

When to use

- SSH, custom TCP protocols, WebSockets over raw TCP, gRPC-like setups that don't play well with the proxy
 - Anything requiring long-lived connections without the proxy timeout behavior
-

4) Choosing between HTTP vs TCP for your “UI of the server”

Most “UI in browser” cases should use HTTP Proxy unless you specifically need:

- very long requests without streaming
- custom protocols

- fixed behavior outside of HTTP semantics

If you do need long inference, common patterns are:

- Use HTTP proxy for the UI

Runpod networking “schematic” for building an app that opens a server UI

Mental model

A Pod is your container running on rented compute. Your app listens on an internal port (inside the container), and you choose how Runpod exposes it publicly.

You'll mainly use two exposure methods:

A) HTTP Proxy (recommended for web UIs + REST)

Network path

Browser/App → Cloudflare → Runpod proxy/
load balancer → Pod internal port

Configuration

- Expose an internal port as <port>/http (example: 3000/http)
- Your service must bind: [0.0.0.0](#):<port> (not [127.0.0.1](#))

Stable URL format

https://POD_ID-INTERNAL_PORT.proxy.runpod.net

Example:

<https://abc123xyz-3000.proxy.runpod.net>

Important behaviors/limits (affects your app design)

- HTTPS is automatic (good for UIs).
- Proxy path has a ~100-second timeout (Cloudflare). If the request doesn't start responding/streaming within that window, it can fail (often seen as 524).

When to use

- Web UIs (React/Next.js apps, Gradio,

Streamlit, etc.)

- REST APIs that respond quickly or stream output

B) Direct TCP (for raw TCP, SSH, long-lived connections)

Network path

Client → Public IP + mapped external port →
Pod internal TCP port

Configuration

- Expose an internal port as <port>/tcp (example: 22/tcp, 8765/tcp)
- Runpod assigns:

- a public IP
- an external port mapped to your internal port
- You view this in the Pod Connect menu (or via Pod details API/SDK).

Key gotcha

The external port is usually different from your internal port, so your app must discover the mapping—don't assume external == internal.

Programmatic discovery (recommended for “Open UI” buttons, clients, etc.)

Runpod provides environment variables like:

- RUNPOD_PUBLIC_IP
- RUNPOD_TCP_PORT_*<internalPort>*

Example: if you exposed 22/tcp, you'll typically see `RUNPOD_TCP_PORT_22`.

When to use

- SSH / IDE remote connections
- Custom TCP protocols
- WebSockets or services that don't work well behind an HTTP proxy
- Very long-lived connections (where proxy timeout is an issue)

How to choose for a “UI of the server”

For “open this UI in my browser,” use HTTP Proxy

unless you specifically need:

- long non-streaming requests that might exceed the proxy timeout
- a protocol that isn't HTTP

A common architecture:

- UI over HTTP proxy
- long work handled as background jobs + polling/streaming progress

What your app should store/display (a clean connection model)

For each service you run inside a pod, model something like:

- podId
- serviceName (e.g. “ComfyUI”, “My Admin UI”)
- internalPort
- exposureType = http_proxy | direct_tcp

Then:

If http_proxy

- url = https://{podId}-
{internalPort}.proxy.runpod.net

If direct_tcp

- publicIp = (from pod details or
RUNPOD_PUBLIC_IP)

- externalPort =
RUNPOD_TCP_PORT_<internalPort>
- address = {publicIp}:{externalPort}

This avoids hardcoding and lets your app render an “Open UI” / “Copy address” action reliably.

Optional: Private pod-to-pod networking (no public ports)

If you need services to talk privately between pods without opening ports to the internet, Runpod has Global Networking (Secure Cloud, supported data centers). Pods get an internal hostname on a *.runpod.internal network.

Sources

- Expose ports (proxy URL, Direct TCP, timeout, env vars): <https://docs.runpod.io/pods/configuration/expose-ports>
- Integrate with Pods (proxy + direct TCP overview): <https://docs.runpod.io/integrations/overview>
- Proxy behavior + tradeoffs: <https://www.runpod.io/blog/runpod-proxy-guide>
- Global Networking overview: <https://www.runpod.io/blog/global-networking-cross-datacenter-pod-communication>

Two questions to tailor this to your exact use case

1. Is your “server UI” running on a Pod (always-

on) or a Serverless endpoint (job-based autoscaling)?

2. What UI/framework and port (e.g. Next.js on 3000, Gradio on 7860, Streamlit on 8501)?